JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class ByteArrayInputStream

java.lang.Object
    java.io.InputStream
        java.io.ByteArrayInputStream

All Implemented Interfaces:

`Closeable, AutoCloseable`

```
public class ByteArrayInputStream
extends InputStream
```

A `ByteArrayInputStream` contains an internal buffer that contains bytes that may be read from the stream. An internal counter keeps track of the next byte to be supplied by the `read` method.

Closing a `ByteArrayInputStream` has no effect. The methods in this class can be called after the stream has been closed without generating an `IOException`.

Since:

`JDK1.0`

See Also:

`StringBufferInputStream`

Field Summary

Fields

Modifier and Type	Field	Description
`protected byte[]`	**`buf`**	An array of bytes that was provided by the creator of the stream.

protected int	**count**	The index one greater than the last valid character in the input stream buffer.
protected int	**mark**	The currently marked position in the stream.
protected int	**pos**	The index of the next character to read from the input stream buffer.

Constructor Summary

Constructors

Constructor	Description
ByteArrayInputStream(byte[] buf)	Creates a ByteArrayInputStream so that it uses buf as its buffer array.
ByteArrayInputStream(byte[] buf, int offset, int length)	Creates ByteArrayInputStream that uses buf as its buffer array.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
int	**available**()	Returns the number of remaining bytes that can be read (or skipped over) from this input stream.
void	**close**()	Closing a ByteArrayInputStream has no effect.
void	**mark**(int readAheadLimit)	Set the current marked position in the stream.
boolean	**markSupported**()	Tests if this InputStream supports mark/reset.

int	**read**()	Reads the next byte of data from this input stream.
int	**read**(byte[] b, int off, int len)	Reads up to len bytes of data into an array of bytes from this input stream.
void	**reset**()	Resets the buffer to the marked position.
long	**skip**(long n)	Skips n bytes of input from this input stream.

Methods inherited from class java.io.InputStream

read

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

buf

```
protected byte[] buf
```

An array of bytes that was provided by the creator of the stream. Elements `buf[0]` through `buf[count-1]` are the only bytes that can ever be read from the stream; element `buf[pos]` is the next byte to be read.

pos

```
protected int pos
```

The index of the next character to read from the input stream buffer. This value should always be nonnegative and not larger than the value of `count`. The next byte to be read from the input stream buffer will be `buf[pos]`.

mark

```
protected int mark
```

The currently marked position in the stream. ByteArrayInputStream objects are marked at position zero by default when constructed. They may be marked at another position within the buffer by the `mark()` method. The current buffer position is set to this point by the `reset()` method.

If no mark has been set, then the value of mark is the offset passed to the constructor (or 0 if the offset was not supplied).

Since:

`JDK1.1`

count

```
protected int count
```

The index one greater than the last valid character in the input stream buffer. This value should always be nonnegative and not larger than the length of `buf`. It is one greater than the position of the last byte within `buf` that can ever be read from the input stream buffer.

Constructor Detail

ByteArrayInputStream

```
public ByteArrayInputStream(byte[] buf)
```

Creates a `ByteArrayInputStream` so that it uses `buf` as its buffer array. The buffer array is not copied. The initial value of `pos` is `0` and the initial value of `count` is the length of `buf`.

Parameters:

`buf - the input buffer.`

ByteArrayInputStream

```
public ByteArrayInputStream(byte[] buf,
                            int offset,
                            int length)
```

Creates `ByteArrayInputStream` that uses `buf` as its buffer array. The initial value of `pos` is `offset` and the initial value of `count` is the minimum of `offset+length` and `buf.length`. The buffer array is not copied. The buffer's mark is set to the specified offset.

Parameters:

`buf - the input buffer.`

`offset - the offset in the buffer of the first byte to read.`

`length - the maximum number of bytes to read from the buffer.`

Method Detail

read

```
public int read()
```

Reads the next byte of data from this input stream. The value byte is returned as an `int` in the range `0` to `255`. If no byte is available because the end of the stream has been reached, the value `-1` is returned.

This `read` method cannot block.

Specified by:

`read` in class `InputStream`

Returns:

`the next byte of data, or -1 if the end of the stream has been reached.`

read

```
public int read(byte[] b,
                int off,
                int len)
```

Reads up to `len` bytes of data into an array of bytes from this input stream. If `pos` equals `count`, then `-1` is returned to indicate end of file. Otherwise, the number `k` of bytes read is equal to the smaller of `len` and `count-pos`. If `k` is positive, then bytes `buf[pos]` through `buf[pos+k-1]` are copied into `b[off]` through `b[off+k-1]` in the manner performed by `System.arraycopy`. The value `k` is added into `pos` and `k` is returned.

This `read` method cannot block.

Overrides:

`read` in class `InputStream`

Parameters:

`b - the buffer into which the data is read.`

`off - the start offset in the destination array b`

`len - the maximum number of bytes read.`

Returns:

`the total number of bytes read into the buffer, or -1 if there is no more data because the end of the stream has been reached.`

Throws:

`NullPointerException - If b is null.`

`IndexOutOfBoundsException - If off is negative, len is negative, or len is greater than b.length - off`

See Also:

`InputStream.read()`

skip

```
public long skip(long n)
```

Skips `n` bytes of input from this input stream. Fewer bytes might be skipped if the end of the input stream is reached. The actual number `k` of bytes to be skipped is equal to the smaller of `n` and `count-pos`. The value `k` is added into `pos` and `k` is returned.

Overrides:

`skip in class InputStream`

Parameters:

`n - the number of bytes to be skipped.`

Returns:

`the actual number of bytes skipped.`

available

```
public int available()
```

Returns the number of remaining bytes that can be read (or skipped over) from this input stream.

The value returned is `count - pos`, which is the number of bytes remaining to be read from the input buffer.

Overrides:

`available in class InputStream`

Returns:

the number of remaining bytes that can be read (or skipped over) from this input stream without blocking.

markSupported

```
public boolean markSupported()
```

Tests if this `InputStream` supports mark/reset. The `markSupported` method of `ByteArrayInputStream` always returns `true`.

Overrides:

`markSupported` in class `InputStream`

Returns:

`true` if this stream instance supports the mark and reset methods; `false` otherwise.

Since:

JDK1.1

See Also:

`InputStream.mark(int)`, `InputStream.reset()`

mark

```
public void mark(int readAheadLimit)
```

Set the current marked position in the stream. ByteArrayInputStream objects are marked at position zero by default when constructed. They may be marked at another position within the buffer by this method.

If no mark has been set, then the value of the mark is the offset passed to the constructor (or 0 if the offset was not supplied).

Note: The `readAheadLimit` for this class has no meaning.

Overrides:

`mark` in class `InputStream`

Parameters:

`readAheadLimit` - the maximum limit of bytes that can be read before the mark position becomes invalid.

Since:

JDK1.1

See Also:

`InputStream.reset()`

reset

```
public void reset()
```

Resets the buffer to the marked position. The marked position is 0 unless another position was marked or an offset was specified in the constructor.

Overrides:

`reset` in class `InputStream`

See Also:

`InputStream.mark(int)`, `IOException`

close

```
public void close()
           throws IOException
```

Closing a `ByteArrayInputStream` has no effect. The methods in this class can be called after the stream has been closed without generating an `IOException`.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Overrides:

`close` in class `InputStream`

Throws:

`IOException` - if an I/O error occurs.

contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.